

Ensemble Methods for Collaborative Intelligence: Implementing 8 Methods to Combine ML Models Trained on Distributed, Ubiquitous IoT Devices

Bharath Sudharsan^{*}, Duc-Duy Nguyen^{*}, John Breslin

^{*}Data Science Institute
The National University of Ireland Galway
IDA Business Park, Lower Dangan, Galway, Ireland
H91AEX4, Ireland
{b.sudharsan1, d.nguyen4, john.breslin}@nuigalway.ie

Abstract

Following the acceleration in the complication of contemporary Artificial Intelligence problems, A single one-size-fits-all Machine Learning model exposed to be frail and insufficient to tackle such complexity. Intuitively, Integrating the strength of multiple models toward the same purpose is a prominent approach to resolve such problem. This paper demonstrate an effective method to combine multiple light-weight ML models trained on various IoT devices. The proposed method exemplify how a complicated collective intelligence can be derived from numerous elementary intelligences learned by Internet of things(IoT) devices

Introduction

Nowadays, in many real-world use cases, the concept of combining models has gained much attention since it boosts performance while maintaining stability. In a non-ideal scenario, there are chances for a base model to make independent but complementary errors. In such cases, to achieve more reliable results, multiple base models need to be combined using an algorithm/method to form a central model that eliminates the builtin randomness and uncertainties. Combining models have other advantages, as we explained earlier, using the *providing sensitive medical data for research* use case. Similarly, in the Federated Learning domain, collaborative learning is accomplished without exchanging raw data among IoT devices by *combining models rather than combining distributed data*. When the combined central model is flashed on the distributed edge devices via an OTA update, each device can attain a global view, resulting in improved intelligence, thus providing more accurate edge-analytics result.

To address the above privacy and cyber-security issues, we need to follow the *model aggregation rather than the data aggregation approach*. To implement this approach, we first need an algorithm that can comfortably execute (without disturbing device routine) on resources-constrained IoT edge devices (where data is generated) to train models offline, even without temporarily storing the live use case data in the device memory. Then, the resultant models that were trained using decentralized data should be combined in a central location. In the following, we present an example

use case to show how our *Train++* can be used in such a collaborative learning/knowledge sharing scenario.

Applications and Use Cases

Providing sensitive medical data for research

The data required for most researches are sensitive in nature, as it revolves around on a private individual. Currently, privacy regulations restrict sending sensitive yet valuable medical data from hospitals and imaging centers to the biological research conducting pharmaceutical companies. As shown in Fig. 1, when the resource-constrained medical devices like insulin-delivery devices, electric steam sterilizers, BP apparatus, etc., are equipped with *Train++*, they will become capable of training offline using the sensitive medical data, even without depending on the hospital's local servers. After training, we can extract the weights (learned information) of the trained models from multiple similar devices, without exposing the data and send it to research firms (securely via https) without voiding the privacy regulations. This method of transmitting trained models (knowledge gained from distributed data) instead of the actual data, fuels fine element analysis at various stages of drug development, from identifying target molecules to recruiting patients for clinical trials by providing the much needed sensitive data. Also, enables manufactures to know their device field performance, perform analysis for early warnings, etc.

Machine Learning Model Combining

In our scenario, we have 10 local models for one dataset (i.e., one device had produced one model), using which we aim to create a central model for each selected dataset. For combining the models, we chose 8 robust methods that apply to a wide range of IoT use-case data while also suitable for combining models trained on devices from different platforms (i.e., MCUs and CPUs). In the remainder of this section, we outline the chosen combining methods [Zhou 2012, Woods, Kegelmeyer, and Bowyer 1997, Ko, Sabourin, and Britto 2008, a20 2016], implement them on our models from section ?? to obtain 3 central models. After combining, we investigate the performance of resultant central models and also the combination methods.

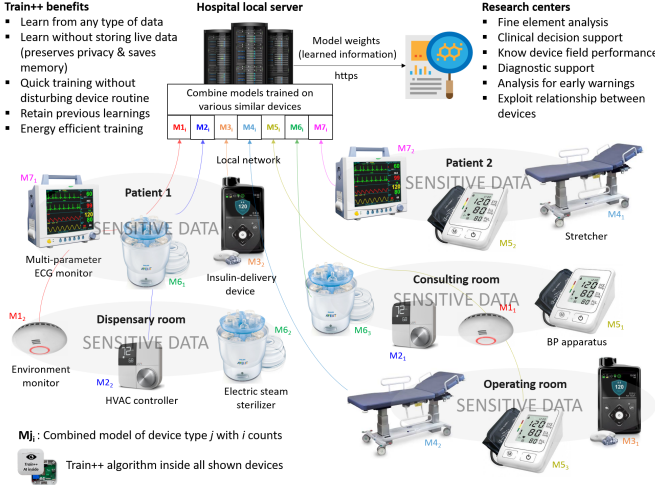


Figure 1: Collaborative learning among IoT devices: A use case of privacy-preserving sensitive medical data collection.

Simple Averaging

We start with this most fundamental method, where we obtained the combined output by directly averaging the outputs of individual base classifiers. When we are given a set of wT individual base classifiers $\{h_1, \dots, h_T\}$ and the output of h_i for the input x is $h_i(x) \in \mathbb{R}$, our task becomes to combine all the h_i to attain the final prediction on the real-valued variable.

Due to its effectiveness and simplicity, this method is represented as the first choice for many real-world applications. However, the error reduction method of simple averaging is derived based on the assumption that the errors of the individual base classifiers are uncorrelated, while in our case of strategically combining classifiers, the errors are typically highly correlated since the base classifiers are trained to solve the same problem. Therefore, the error reduction method of simple averaging, which suggests that the *ensemble error is smaller by a factor of T (value of total base classifiers) than the averaged error of the individual classifiers* is generally hard to achieve.

Weighted Averaging

Here, we obtain the combined output by averaging the outputs of individual learners with different weights implying different importance. The above simple averaging which can be regarded as taking equal weights for all individual learners can be viewed as a special case of weighted averaging. Similarly, other combination methods such as voting are also special cases or variants of weighted averaging.

The data in real tasks are usually noisy and insufficient, resulting in producing estimated weights that are often unreliable. Particularly when combining a large number of classifiers, more weights need to be learned, which can easily lead to overfitting. Simple averaging does not have to learn any weights, so it suffers little from overfitting. In general, we recommend using the above simple averaging for combining classifiers with similar performances, whereas if the

individual classifiers exhibit nonidentical strength, weighted averaging with unequal weights can show better performance.

Alter title: Voting

Voting. The concept of voting is the fundamental and most popular combining method for nominal outputs (classifiers). Since we are dealing with combining classifiers, we use a classification example to explain this concept. When we have T number of individual classifiers $\{h_1, \dots, h_T\}$, here, our task is to combine all h_i to predict the class label from a set of l possible class labels. Since our *Train++* classifiers are trained to solve binary problems, the class labels are $\{c_0, c_1\}$. Here, for an input x , the outputs of the classifier h_i are given an l dimensional label vector $(h_i^1(x), \dots, h_i^l(x))^T$, where $h_i^j(x)$ is the output of h_i , for the class label c_j . The $h_i^j(x)$ takes different types of values according to the information provided by the individual classifiers, e.g., for a *Crisp label* (our case): $h_i^j(x) \in \{0, 1\}$, which takes value 1 if h_i predicts c_j as the class label and 0 otherwise. Or can take *Class probability*: $h_i^j(x) \in [0, 1]$, which can be regarded as an estimate of the posterior probability $P(c_j|x)$.

Majority Voting. The most popular voting method is the *Majority Voting*. Here, each of our 10 base classifiers votes for one class label, and the final output class label is the one that receives more than half of the votes. If none of the class labels receives more than half of the votes, a rejection option will be given and the combined classifier makes no prediction. Hence the output class label of the combined central classifier becomes,

$$H(x) = \begin{cases} c_j & \text{if } \sum_{i=1}^T h_i^j(x) > \frac{1}{2} \sum_{k=1}^l \sum_{i=1}^T h_i^k(x), \\ \text{reject} & \text{otherwise.} \end{cases} \quad (1)$$

This suits well when the individual classifiers are statistically independent, but since all the base classifiers are trained to solve the same problem (i.e., the same dataset is used), they are highly correlated. Therefore, it is impractical to expect the majority voting accuracy to converge to one when the base classifier count is high.

Weighted Majority Voting. In reality, each base classifier is reliable in different areas of the problem/dataset, and most times, the individual classifiers are with unequal performance. Hence, it is highly inefficient to treat the labels from different classifiers with the same weight. So in the *Weighted Majority Voting* (WMV) method, which is the natural extension of above majority voting, we give more power to the stronger classifiers in voting. Hence, as shown in the Eqn (2), the weighted majority voting rule is an extension of Eqn (1) with a weight $w_i \geq 0$ assigning to the classifier h_i .

$$H(x) = c_{\arg \max} \sum_{i=1}^T w_i h_i^j(x). \quad (2)$$

Here, the accuracy of the final aggregated label depends much on the accuracy of the estimated weights. During the distributed model training, some of the IoT devices involved

might be from a new vendor who had followed a new type of MCU architecture, unlike the widely adopted RISC, and/or they might be fed with low-quality data (noisy). On such devices/data, our *Train++* or other training algorithms might produce models with varying results. In such cases, WMV is more suitable to make the devices with a higher competence level counted more for the final decision.

Maximization

There are other combination methods in addition to the ones we explained so far. The maximization and the median methods are the *algebraic methods* we implemented next, that we brief here.

The class probability output from individual base classifiers trained on different IoT devices can be considered as an estimate of the posterior probabilities. Hence, it is straightforward to derive combination rules under the probabilistic framework. The $h_i^j(x)$ is the class probability of c_j output from h_i as h_i^j . According to Bayesian decision theory, given T classifiers, the input x from the dataset should be assigned to the class c_j , which maximizes the posterior probability $P(c_j|h_1^j, \dots, h_T^j)$. Since h_i^j is the probability output, we have $P(c_j|h_i^j) = h_i^j$. Thus, if all classes are with equal priority, we get the product rule for combination as

$$H^j(x) = \prod_{i=1}^T h_i^j(x). \quad (3)$$

Similarly to the above, the maximum/minimum/median rules were derived using which here, we choose the *maximum* and *median* of the individual outputs as the combined output. For example, the median rule generates the combined output according to $H^j(x) = \text{med}(h_i^j(x))$, where $\text{med}(\cdot)$ denotes the median statistic. In short, the maximization method is carried out by taking the maximum scores, and in **Combine by Median** method, the median value across all scores/prediction results are taken.

Dynamic Classifier Selection

We also implement the Dynamic Classifier Selection by Local Accuracy (DCS-LA) and use it on our models. Here, each base classifier's accuracy in *local regions* of feature space is estimated, i.e., the best classifier for each partition of the dataset is determined. For classification results (inference), we use the output produced by the most locally accurate classifier as the final decision. Out of the two methods presented in DCS-LA, we choose to estimate the local accuracy w.r.t. some output class. In other words, when considering a classifier that assigns an input data sample to class c_i , we can determine the percentage of the local training samples assigned to class c_i by this classifier that has been correctly labeled.

Dynamic Ensemble Selection

In most use cases like handwritten pattern recognition, multiple classifiers are used to improve the recognition rates. To optimize such a multiple classifier system, a group of

classifiers, known as an Ensemble of Classifiers (EoC), are selected from a pool of classifiers. To achieve high performance, dynamic selection methods (select different classifiers for different data patterns) need to be used over static selection methods (select an EoC for all data patterns). To combine models from various IoT devices, as introduced, we implement a scheme that dynamically selects an ensemble (instead of dynamic classifier selection, dynamic ensemble selection is performed here) for every test data point. We implemented this method because it can perform better than static ensemble selection methods, and also, using an ensemble of classifiers provides more stability than a single classifier.

Stacking (Meta Ensembling)

In this method, we combine information from multiple base classifier models to generate a new model. Thus generated 2nd level model (stacked model) often outperforms base models due to its smoothing nature and its ability to identify each base classifier where it performs best and discredit it in the areas where it performs poorly. We perform stacking in two phases. Firstly, the base classifiers are trained across various MCUs (uses our *Train++*) and CPUs using a dataset. Then in a central location, the output of each model (i.e., meta-features) is collected to build a new dataset. So, the stacked model can discern where each model performs well and where each model performs poorly. Secondly, this new dataset is used with any meta-learning algorithm to provide the final classification result. Choosing the right stacker and the features is *more of an art than science*.

Performance Analysis

In the remainder of this section, using the information from Fig. 2 a-c (y-axis in log scale), we analyze the performance of the above combining techniques in terms of Accuracy, ROC, and F1 score (F1) evaluation metrics. For the Banknote Authentication dataset, the highest values for metrics are shown by the Dynamic Classifier Selection (DCS-LA) method, followed by Maximization and combine by Median methods, where both show the same accuracy and slightly different ROC and F1. The Simple Averaging, Weighted Averaging, and the Weighted Majority Vote (WMV) methods achieve very similar performance. The combine by Stacking is the least performing, followed by Dynamic Ensemble Selection (DES) method.

For Haberman's Survival dataset again, DCS-LA showed the top performance. The DES and Stacking methods that produced a low performance for the previous dataset are the second and third best-performing methods. The other algebraic, averaging, and voting methods perform almost the same, achieving good accuracy and F1 but low ROC.

For the Titanic dataset, Stacking shows the highest accuracy, but DES achieved slightly higher ROC and F1 so, DES is the overall top-performing method. Unlike in previous datasets, here, the algebraic, averaging, and voting methods show varying performance. From the algebraic methods, the combining by Median performed better, and from averaging methods, Simple Averaging performed better.

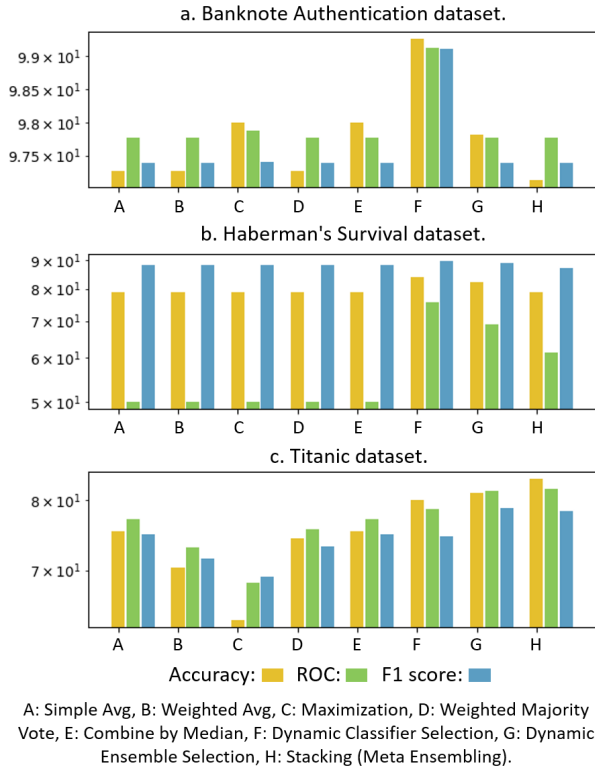


Figure 2: Applying various techniques to combine multiple models trained on MCUs 1-5 and CPUs 1-5. A central model is created for each dataset.

We conclude this section sharing our experience during the experiments; (i) We noticed the computational cost for creating an ensemble is not much greater than training a single base model because multiple versions of the base model need to be generated for model selection or parameter tuning. Also, the computational cost for combining base models was small due to the simplicity of the strategies. (ii) To construct a good ensemble, we recommend the users to create base models *as accurate and as diverse as possible*. (iii) Creating a learning algorithm that is consistently better than others is a hopeless daydream. i.e., from Fig. 2, Stacking shows top performance for the Titanic dataset and least in the Banknote dataset.

Conclusion & Future work

References

2016. Guide to Model Stacking (i.e. Meta Ensembling). URL <https://www.gormanalysis.com/blog/guide-to-model-stacking-i-e-meta-ensembling/>.
- Ko, A.; Sabourin, R.; and Britto, A. S. 2008. From dynamic classifier selection to dynamic ensemble selection. *Pattern Recognit.* .
- Woods, K.; Kegelmeyer, W. P.; and Bowyer, K. 1997. Combination of multiple classifiers using local accuracy estimates. *IEEE Transactions on Pattern Analysis and Machine Intelligence* 19(4): 405–410. doi:10.1109/34.588027.
- Zhou, Z.-H. 2012. *Ensemble Methods: Foundations and Algorithms*. Chapman Hall/CRC, 1st edition. ISBN 1439830037.